

TERRAFORM

Multiple Region -

Creating infrastructure in **more than one AWS region** using the same Terraform code.

You can make use of `alias` keyword to implement multi region infrastructure setup in terraform.

```
provider "aws" {  
  alias = "us-east-1"  
  region = "us-east-1"  
}  
  
provider "aws" {  
  alias = "us-west-2"  
  region = "us-west-2"  
}  
  
resource "aws_instance" "example" {  
  ami = "ami-0123456789abcdef0"  
  instance_type = "t2.micro"  
  provider = "aws.us-east-1"  
}  
  
resource "aws_instance" "example2" {  
  ami = "ami-0123456789abcdef0"  
  instance_type = "t2.micro"  
  provider = "aws.us-west-2"  
}
```

We are creating resources in **multiple AWS regions** using **one Terraform project**

Example:

- Region 1 → us-east-1
- Region 2 → us-west-2

Step 1: Define First Provider (Region 1)

```
provider "aws" {  
  alias = "us-east-1"  
  region = "us-east-1"  
}
```

Explanation

- `provider "aws"` → Using AWS
- `alias = "us-east-1"` → Give this provider a **name**
- `region = "us-east-1"` → Region where resources will be created

This creates a **named AWS provider for us-east-1**

Step 2: Define Second Provider (Region 2)

```
provider "aws" {
  alias   = "us-west-2"
  region = "us-west-2"
}
```

Explanation

- Another AWS provider
- Different alias → `us-west-2`
- Different region

Now you have **two providers**

IMPORTANT -Terraform now knows:

Alias	Region
<code>us-east-1</code>	<code>us-east-1</code>
<code>us-west-2</code>	<code>us-west-2</code>

Step 3: Create Resource in Region 1

```
resource "aws_instance" "example" {
  ami           = "ami-0123456789abcdef0"
  instance_type = "t2.micro"
  provider      = aws.us-east-1
}
```

Explanation

- resource `"aws_instance"` → EC2 instance
- provider = `aws.us-east-1` → Use provider with alias `us-east-1`

This EC2 will be created in **us-east-1**

Step 4: Create Resource in Region 2

```
resource "aws_instance" "example2" {
  ami           = "ami-0123456789abcdef0"
  instance_type = "t2.micro"
  provider      = aws.us-west-2
}
```

Explanation

- Uses second provider
- provider = `aws.us-west-2`

This EC2 will be created in **us-west-2**

Step 5: Run Terraform

```
terraform init
terraform plan
terraform apply
```

Multiple Providers -

Providers = plugins Terraform uses to talk to services (AWS, Azure, GCP, etc.)

Multiple providers means: Using **more than one provider** OR using **same provider multiple times with different configs**

You can use multiple providers in one single terraform project. For example,

1. Create a providers.tf file in the root directory of your Terraform project.
2. In the providers.tf file, define the AWS and Azure providers. For example:

```
provider "aws" {  
    region = "us-east-1"  
}  
  
provider "azurerm" {  
    subscription_id = "your-azure-subscription-id"  
    client_id       = "your-azure-client-id"  
    client_secret   = "your-azure-client-secret"  
    tenant_id       = "your-azure-tenant-id"  
    features {}  
}
```

Write Provider Code

You create this file (usually `providers.tf`)

What you are doing:

- Telling Terraform: "I want to use AWS and Azure"

AWS Provider Setup

```
provider "aws" {  
  
    region = "us-east-1"  
}
```

What happens:

1. Terraform reads `provider "aws"`
2. It understands:
 - You want AWS resources
3. It stores:
 - `Region = us-east-1`

Result: All AWS resources will go to **us-east-1**

Azure Provider Setup

```
provider "azurerm" {  
    subscription_id = "..."
```

```

client_id      = "...
client_secret  = "...
tenant_id     = "...
features {}
}

```

What happens:

1. Terraform reads provider "azurerm"
2. It prepares connection to Azure
3. Uses credentials to authenticate

Each field meaning:

- subscription_id → Your Azure account
- client_id → App ID (like username)
- client_secret → Password
- tenant_id → Organization ID
- features {} → Required block (must exist)

Result: Terraform can now create **Azure resources**

3. In your other Terraform configuration files, you can then use the aws and azurerm providers to create resources in AWS and Azure, respectively,

```

resource "aws_instance" "example" {
  ami = "ami-0123456789abcdef0"
  instance_type = "t2.micro"
}

resource "azurerm_virtual_machine" "example" {
  name = "example-vm"
  location = "eastus"
  size = "Standard_A1"
}

```

Terraform Reads the Resource Blocks

When you run: terraform plan / apply

What Terraform sees:

2 resources:

1. aws_instance → AWS EC2
2. azurerm_virtual_machine → Azure VM

It prepares to create both

Step 2: First Resource (AWS EC2)

```

resource "aws_instance" "example" {

```

resource

- Keyword to create infrastructure

"aws_instance"

- Type of resource
- Means: **EC2 instance in AWS**

"example"

- Name (local reference)
- Used inside Terraform like:

`aws_instance.example.id`

Inside the Block

```
ami = "ami-0123456789abcdef0"
```

What is AMI?

- AMI = Amazon Machine Image
- It defines:
 - OS (Linux/Windows)
 - Pre-installed software

Example: Amazon Linux

```
instance_type = "t2.micro"
```

What is instance type?

- Defines:
 - CPU
 - RAM

`t2.micro` = Free tier eligible (small machine)

What Happens Internally

1. Terraform selects **AWS provider**
2. Connects to AWS API
3. Sends request:

“Create EC2 with this AMI and type”

Result

EC2 instance is created in AWS

Step 3: Second Resource (Azure VM)

```
resource "azurerm_virtual_machine" "example" {
```

- ♦ **"azurerm_virtual_machine"**
 - Resource type
 - Means: **Virtual Machine in Azure**

- ♦ **"example"**
 - Name inside Terraform
 - Separate from AWS (no conflict)

♦ **Inside the Block**

`name = "example-vm"`

Meaning:

- Name of VM in Azure portal

`location = "eastus"`

Meaning:

- Azure region
- Example: East US data center

`size = "Standard_A1"`

Meaning:

- VM size (CPU + RAM)
- Similar to AWS instance type

What Happens Internally

1. Terraform selects **Azure provider**
2. Connects to Azure API
3. Sends request: "Create VM with this name, region, and size"

Step 4: How Terraform Chooses Provider

Resource	Provider Used
<code>aws_instance</code>	AWS
<code>azurerm_virtual_machine</code>	Azure
Automatic selection based on type	

Step 5: Execution Flow

Run: `terraform init`

Downloads AWS + Azure plugins

Run: terraform plan

Shows:

```
+ create aws_instance.example
+ create azurerm_virtual_machine.example
```

Run: terraform apply

What happens:

- AWS → EC2 created
- Azure → VM created

Variables

Terraform variables are used to make your configuration dynamic, reusable, and flexible. Instead of hardcoding values (like region, instance type, etc.), you can define variables and pass different values when needed.

Variables Demo

```
```hcl
```

# Define an input variable for the EC2 instance type

```
variable "instance_type" {
 description = "EC2 instance type"
 type = string
 default = "t2.micro"
}
```

# Define an input variable for the EC2 instance AMI ID

```
variable "ami_id" {
 description = "EC2 AMI ID"
 type = string
}
```

# Configure the AWS provider using the input variables

```

provider "aws" {
 region = "us-east-1"
}

Create an EC2 instance using the input variables
resource "aws_instance" "example_instance" {
 ami = var.ami_id
 instance_type = var.instance_type
}

Define an output variable to expose the public IP address of the EC2 instance
output "public_ip" {
 description = "Public IP address of the EC2 instance"
 value = aws_instance.example_instance.public_ip
}

...

```

## 1) Input Variables

Input variables in Terraform are used to make your code dynamic and reusable. Instead of hardcoding values, you can pass them from outside.

It is like a **parameter** that allows you to give values to your Terraform configuration.

For Example -

```

variable "example_var" {
 description = "An example input variable"
 type = string
 default = "default_value"
}

```

### Step 1 – variable "example\_var"

This declares an **input variable**

- "example\_var" → Name of the variable



- You will use it as:

```
var.example_var
```

## Step 2 – description

```
description = "An example input variable"
```

This is just **documentation**

- Helps explain what the variable is used for
- Useful for readability and team understanding

## Step 3 – type

```
type = string
```

Defines the **data type**

- `string` → Accepts text values
- Ensures correct input is provided

Other types:

- `number`
- `bool`
- `list`
- `map`
- `object`

## Step 4 – default

```
default = "default_value"
```

Provides a **default value**

- If no value is given → Terraform uses "default\_value"
- Makes the variable **optional**

## Step 5 – How It Works

### Case 1: No value provided

```
terraform apply
```

Uses default:

```
default_value
```

### Case 2: Value provided

```
terraform apply -var="example_var=custom_value"
```

Overrides default:

```
custom_value
```

## Step 6 – Usage in Resource

```
resource "example_resource" "demo" {
 name = var.example_var
}
```

The resource will use:

- "default\_value" OR
- "custom\_value" (if provided)

\*You can then use the input variable within your module or configuration like this:

```
resource "example_resource" "example" {
 name = var.example_var
 # other resource configurations
}
```

This code **creates a resource** and sets its **name using a variable**

### 1. resource "example\_resource" "example"

- Creates something in Terraform (like EC2, S3, etc.)
- "example\_resource" → type
- "example" → name

### 2. name = var.example\_var

- Sets the **name of the resource**
- Value comes from a **variable**

var.example\_var = value you give from outside

## How value is decided

### Case 1: Default value used

If variable has:

```
default = "default_value"
```

Then:

```
name = "default_value"
```

### Case 2: You pass value

```
terraform apply -var="example_var=my-app"
```

Then:

```
name = "my-app"
```

## Easy Understanding

Instead of writing fixed value:

```
name = "my-server"
```

You use variable:

```
name = var.example_var
```

So you can change name anytime

No need to edit code

## 2. Output Variables

Output variables in Terraform are used to display or expose values from your infrastructure after it is created.

Here's how you define an output variable:

```
output "example_output" {
 description = "An example output variable"
 value = resource.example_resource.example.id
}
```

### Step 1 – output "example\_output"

This defines an **output variable name**

- "example\_output" → The name of the output
- This name will be shown after running Terraform

Example result:

```
example_output = some-value
```

### Step 2 – description

```
description = "An example output variable"
```

This is just **documentation**

- Helps users understand what this output is
- Not required, but recommended

### Step 3 – value

```
value = example_resource.example.id
```

This is the **actual value Terraform will display**

- example\_resource → Resource type
- example → Resource name
- .id → Attribute of that resource

format: <resource\_type>.<resource\_name>.<attribute>

## Step 4 – What Happens When You Run Terraform

After: terraform apply

Terraform will display: **example\_output = <resource-id>**

Example: example\_output = i-0abc123456789xyz

\* You can reference output variables in the root module or in other modules by using the syntax `module.module_name.output_name`, where `module_name` is the name of the module containing the output variable.

For example, if you have an output variable named `example_output` in a module called `example_module`, you can access it in the root module like this:

```
output "root_output" {
 value = module.example_module.example_output
}
```

## Step 1 – Define Output Name

`output "root_output"`

- Creates an output variable named **root\_output**
- This is what Terraform will print after execution

## Step 2 – Understand the Value

`value = module.example_module.example_output`

Break it into 3 parts:

### 1. module

- Refers to a **Terraform module**

### 2. example\_module

- Name of the module (defined in root)

Example:

```
module "example_module" {
 source = "../my-module"
}
```

### 3. example\_output

- This is an **output defined inside the module**

Inside module:

```
output "example_output" {
 value = aws_instance.example.id
}
```

### Step 3 – What This Line Does

It **fetches the output from the module** and exposes it in the root module

You are basically saying:

“Take the output from the module and show it here”

### Step 4 – Execution Flow

1. Terraform runs the module
2. Module creates resources
3. Module generates `example_output`
4. Root module reads it
5. Displays it as `root_output`

### Step 5 – Final Output Example

After running: `terraform apply`

You will see: `root_output = i-1234567890abcdef`

## # Terraform tfvars

In Terraform, `.tfvars` files (typically with a `.tfvars` extension) are used to set specific values for input variables defined in your Terraform configuration.

They allow you to separate configuration values from your Terraform code, making it easier to manage different configurations for different environments (e.g., development, staging, production) or to store sensitive information without exposing it in your code.

Here's the **purpose of .tfvars files**:

1. **Separation of Configuration from Code:** Input variables in Terraform are meant to be configurable so that you can use the same code with different sets of values. Instead of hardcoding these values directly into your `.tf` files, you use `.tfvars` files to keep the configuration separate. This makes it easier to maintain and manage configurations for different environments.
2. **Sensitive Information:** `.tfvars` files are a common place to store sensitive information like API keys, access credentials, or secrets. These sensitive values can be kept outside the

version control system, enhancing security and preventing accidental exposure of secrets in your codebase.

3. **Reusability:** By keeping configuration values in separate `.tfvars` files, you can reuse the same Terraform code with different sets of variables. This is useful for creating infrastructure for different projects or environments using a single set of Terraform modules.
4. **Collaboration:** When working in a team, each team member can have their own `.tfvars` file to set values specific to their environment or workflow. This avoids conflicts in the codebase when multiple people are working on the same Terraform project.

## Summary

Here's how you typically use `.tfvars` files

1. Define your input variables in your Terraform code (e.g., in a `variables.tf` file).
2. Create one or more `.tfvars` files, each containing specific values for those input variables.
3. When running Terraform commands (e.g., `terraform apply`, `terraform plan`), you can specify which `.tfvars` file(s) to use with the `-var-file` option:

```
terraform apply -var-file=dev.tfvars
```

By using `.tfvars` files, you can keep your Terraform code more generic and flexible while tailoring configurations to different scenarios and environments.

## # Conditional Expressions

Conditional expressions in Terraform are used to define conditional logic within your configurations. They allow you to make decisions or set values based on conditions. Conditional expressions are typically used to control whether resources are created or configured based on the evaluation of a condition.

The syntax for a conditional expression in Terraform is:

### **condition ? true\_val : false\_val**

- `condition` is an expression that evaluates to either `true` or `false`.
- `true_val` is the value that is returned if the condition is `true`.
- `false_val` is the value that is returned if the condition is `false`.

Here are some common use cases and examples of how to use conditional expressions in Terraform:

### **Conditional Resource Creation Example-**

```
resource "aws_instance" "example" {
 count = var.create_instance ? 1 : 0
```

```
ami = "ami-XXXXXXXXXXXXXXXXXXXX"
instance_type = "t2.micro"
}
```

In this example, the `count` attribute of the `aws_instance` resource uses a conditional expression. If the `create_instance` variable is `true`, it creates one EC2 instance. If `create_instance` is `false`, it creates zero instances, effectively skipping resource creation.

### Conditional Variable Assignment Example-

```
variable "environment" {
 description = "Environment type"
 type = string
 default = "development"
}

variable "production_subnet_cidr" {
 description = "CIDR block for production subnet"
 type = string
 default = "10.0.1.0/24"
}

variable "development_subnet_cidr" {
 description = "CIDR block for development subnet"
 type = string
 default = "10.0.2.0/24"
}

resource "aws_security_group" "example" {
 name = "example-sg"
 description = "Example security group"
}
```

```

ingress {
 from_port = 22
 to_port = 22
 protocol = "tcp"

 cidr_blocks = var.environment == "production" ? [var.production_subnet_cidr] :
[var.development_subnet_cidr]
}
}

```

In this example, the `locals` block uses a conditional expression to assign a value to the `subnet_cidr` local variable based on the value of the `environment` variable. If `environment` is set to `"production"`, it uses the `production_subnet_cidr` variable; otherwise, it uses the `development_subnet_cidr` variable.

### Conditional Resource Configuration-

```

resource "aws_security_group" "example" {
 name = "example-sg"
 description = "Example security group"

 ingress {
 from_port = 22
 to_port = 22
 protocol = "tcp"
 cidr_blocks = var.enable_ssh ? ["0.0.0.0/0"] : []
 }
}

```

In this example, the `ingress` block within the `aws_security_group` resource uses a conditional expression to control whether SSH access is allowed. If `enable_ssh` is `true`, it allows SSH traffic from any source (`"0.0.0.0/0"`); otherwise, it allows no inbound traffic.

Conditional expressions in Terraform provide a powerful way to make decisions and customize your infrastructure deployments based on various conditions and variables. They enhance the flexibility and reusability of your Terraform configurations.

### # Built-in Functions-



Terraform is an infrastructure as code (IaC) tool that allows you to define and provision infrastructure resources in a declarative manner. Terraform provides a wide range of built-in functions that you can use within your configuration files (usually written in HashiCorp Configuration Language, or HCL) to manipulate and transform data. These functions help you perform various tasks when defining your infrastructure. Here are some commonly used built-in functions in Terraform:

### **1.concat(list1, list2, ...): Combines multiple lists into a single list.**

```
variable "list1" {
 type = list
 default = ["a", "b"]
}

variable "list2" {
 type = list
 default = ["c", "d"]
}

output "combined_list" {
 value = concat(var.list1, var.list2)
}
```

### **2.element(list, index): Returns the element at the specified index in a list.**

```
variable "my_list" {
 type = list
 default = ["apple", "banana", "cherry"]
}

output "selected_element" {
 value = element(var.my_list, 1) # Returns "banana"
}
```

### **3.length(list): Returns the number of elements in a list.**

```
variable "my_list" {
 type = list
 default = ["apple", "banana", "cherry"]
}

output "list_length" {
 value = length(var.my_list) # Returns 3
}
```

### **4.map(key, value): Creates a map from a list of keys and a list of values.**

```
variable "keys" {
 type = list
 default = ["name", "age"]
}
```

```
variable "values" {
 type = list
 default = ["Alice", 30]
}

output "my_map" {
 value = map(var.keys, var.values) # Returns {"name" = "Alice", "age" = 30}
}
```

### **5. lookup(map, key): Retrieves the value associated with a specific key in a map.**

```
variable "my_map" {
 type = map(string)
 default = {"name" = "Alice", "age" = "30"}
}

output "value" {
 value = lookup(var.my_map, "name") # Returns "Alice"
}
```

### **6. join(separator, list): Joins the elements of a list into a single string using the specified separator.**

```
variable "my_list" {
 type = list
 default = ["apple", "banana", "cherry"]
}

output "joined_string" {
 value = join(", ", var.my_list) # Returns "apple, banana, cherry"
}
```

These are just a few examples of the built-in functions available in Terraform. You can find more functions and detailed documentation in the official Terraform documentation, which is regularly updated to include new features and improvements

## **• Demo 1 – Create EC2 instance using variables, tf.vars , output**

### **Step 1: Provider Configuration (main.tf)**

```
provider "aws" {
 region = "us-east-1"
}
```

#### **What happens here:**

1. Terraform loads AWS provider plugin (during terraform init)
2. Uses your AWS credentials (from CLI / env / config)
3. Sets region → **us-east-1**

### Meaning:

“All resources will be created in this AWS region”

### Step 2: Resource Block (EC2 Creation)

```
resource "aws_instance" "example" {
 ami = var.ami_value
 instance_type = var.instance_type_value
 subnet_id = var.subnet_id_value
}
```

#### What happens step-by-step:

1. Terraform sees resource type → `aws_instance`
2. Resource name → `example`
3. It needs values for:
  - AMI
  - Instance type
  - Subnet
4. Instead of fixed values → it looks for **variables**

### Step 3: Variable Declaration (`variables.tf`)

```
variable "ami_value" {}
variable "instance_type_value" {}
variable "subnet_id_value" {}
```

#### What happens:

1. Terraform registers these as **inputs**
2. It expects values at runtime

### Step 4: Variable Values (`terraform.tfvars`)

```
ami_value = "ami-0ec10929233384c7f"
instance_type_value = "t2.micro"
subnet_id_value = "subnet-01ab6d5eb83572a06"
```

#### What happens:

1. Terraform automatically loads this file
2. Assigns values to variables

### Step 5: Variable Resolution (Important Flow)

```
var.ami_value → "ami-0ec10929233384c7f"
var.instance_type_value → "t2.micro"
var.subnet_id_value → "subnet-xxxx"
```

Now your resource becomes internally:

```
resource "aws_instance" "example" {
 ami = "ami-0ec10929233384c7f"
 instance_type = "t2.micro"
 subnet_id = "subnet-01ab6d5eb83572a06"
```

```
}
```

## Step 6: Output Block

```
output "public-ip-address" {
 value = aws_instance.example.public_ip
}
```

### What happens:

1. After creation, Terraform:
  - Fetches EC2 public IP
2. Prints it on terminal

## Step 7: Commands Execution Flow

### 1. terraform init

```
terraform init
```

#### Internally:

1. Downloads AWS provider
2. Creates .terraform/ folder
3. Prepares environment

### 2. terraform plan

```
terraform plan
```

#### Internally:

1. Reads all .tf files
2. Loads variables
3. Compares with state file
4. Shows:

Plan: 1 to add, 0 to change, 0 to destroy

### 3. terraform apply

```
terraform apply
```

#### Internally:

1. Re-checks plan
2. Calls AWS API:
  - RunInstances
3. Creates EC2
4. Stores info in:

```
terraform.tfstate
```

## Step 8: State File

### File:

terraform.tfstate

### Contains:

- Instance ID
- AMI used
- Public IP
- Resource mapping

### Why needed:

Terraform uses it to track what it created

## Step 9: Full Flow

```
main.tf (resource + provider)
 ↓
variables.tf (declares variables)
 ↓
terraform.tfvars (assigns values)
 ↓
terraform init
 ↓
terraform plan
 ↓
terraform apply
 ↓
AWS EC2 created
 ↓
terraform.tfstate updated
 ↓
Output printed (public IP)
```

## Demo 2- Create EC2 instance using modules

### 1. What is happening in your code

Your file:

```
provider "aws" {
 region = "us-east-1"
}
module "ec2_instance" {
 source = "../module/ec2_instance"
 ami_value = "ami-0ec10929233384c7f"
 instance_type_value = "t2.micro"
 subnet_id_value = "subnet-01ab6d5eb83572a06"
}
```

This is the **ROOT MODULE** (main controller)

## 2. Step-by-Step Execution

### Step 1: Terraform starts

You run:

```
terraform init
terraform apply
```

### Step 2: Provider loads

```
provider "aws" {
 region = "us-east-1"
}
```

Terraform:

1. Downloads AWS provider
2. Connects to AWS
3. Sets region

### Step 3: Module block is detected

```
module "ec2_instance"
```

Terraform understands:

“I need to go inside another folder and execute code there”

### Step 4: Source path

```
source = "../module/ec2_instance"
```

Terraform goes to:

```
module/ec2_instance/
```

### Step 5: Inputs passed to module

```
ami_value = "ami-xxx"
instance_type_value = "t2.micro"
subnet_id_value = "subnet-xxx"
```

These values are sent into the module

## 3. Inside Module Folder

**module/ec2\_instance/**

### variables.tf

```
variable "ami_value" {}
variable "instance_type_value" {}
variable "subnet_id_value" {}
```

Defines inputs module expects

### main.tf (inside module)

```
resource "aws_instance" "example" {
 ami = var.ami_value
 instance_type = var.instance_type_value
 subnet_id = var.subnet_id_value
}
```

Uses values received from root

### outputs.tf

```
output "public-ip-address" {
 value = aws_instance.example.public_ip
}
```

Sends output back to root

## 4. Full Flow

1. terraform apply
- ↓
2. Read root main.tf
- ↓
3. Find module block
- ↓
4. Go to module/ec2\_instance
- ↓
5. Read variables.tf (inputs)
- ↓
6. Receive values from root
- ↓
7. Execute resource block
- ↓
8. Create EC2 in AWS
- ↓
9. Save in terraform.tfstate
- ↓
10. Return output (public IP)

## 5. What You Achieved

You created a **reusable EC2 module**

Instead of writing:

```
resource "aws_instance" {}
```

Again and again

You now use:

```
module "ec2_instance" {}
```

- ✓ Clean
- ✓ Reusable
- ✓ Scalable

## # Module

A Terraform module is a reusable, self-contained collection of Terraform configuration files that define a set of infrastructure resources.”

## Here are the key benefits:

1. **Modularity:** Terraform modules allow you to break down your infrastructure configuration into smaller, self-contained components. This modularity makes it easier to manage and reason about your infrastructure because each module handles a specific piece of functionality, such as an EC2 instance, a database, or a network configuration.
2. **Reusability:** With modules, you can create reusable templates for common infrastructure components. Instead of rewriting similar configurations for multiple projects, you can reuse modules across different Terraform projects. This reduces duplication and promotes consistency in your infrastructure.
3. **Simplified Collaboration:** Modules make it easier for teams to collaborate on infrastructure projects. Different team members can work on separate modules independently, and then these modules can be combined to build complex infrastructure deployments. This division of labor can streamline development and reduce conflicts in the codebase.

Example:

One person → VPC module

Another → EC2 module

4. **Versioning and Maintenance:** Modules can have their own versioning, making it easier to manage updates and changes. When you update a module, you can increment its version, and other projects using that module can choose when to adopt the new version, helping to prevent unexpected changes in existing deployments.
5. **Abstraction:** Modules can abstract away the complexity of underlying resources. For example, an EC2 instance module can hide the details of security groups, subnets, and other configurations, allowing users to focus on high-level parameters like instance type and image ID.

### Example:

Instead of writing:

```
resource "aws_instance" {}
resource "aws_security_group" {}
```

User only uses:

```
module "ec2" {
 instance_type = "t2.micro"
}
```



6. **Testing and Validation:** Modules can be individually tested and validated, ensuring that they work correctly before being used in multiple projects. This reduces the risk of errors propagating across your infrastructure.
7. **Documentation:** Modules promote self-documentation. When you define variables, outputs, and resource dependencies within a module, it becomes clear how the module should be used, making it easier for others (or your future self) to understand and work with.
8. **Scalability:** As your infrastructure grows, modules provide a scalable approach to managing complexity. You can continue to create new modules for different components of your architecture, maintaining a clean and organized codebase.
9. **Security and Compliance:** Modules can encapsulate security and compliance best practices. For instance, you can create a module for launching EC2 instances with predefined security groups, IAM roles, and other security-related configurations, ensuring consistency and compliance across your deployments.

## # Terraform State File

Terraform is an Infrastructure as Code (IaC) tool used to define and provision infrastructure resources. The Terraform state file is a crucial component of Terraform that helps it keep track of the resources it manages and their current state.

This file, often named `terraform.tfstate`, is a JSON or HCL (HashiCorp Configuration Language) formatted file that contains important information about the infrastructure's current state, such as resource attributes, dependencies, and metadata.

### Advantages of Terraform State File:

1. **Resource Tracking:** The state file keeps track of all the resources managed by Terraform, including their attributes and dependencies. This ensures that Terraform can accurately update or destroy resources when necessary.
2. **Concurrency Control:** Terraform uses the state file to lock resources, preventing multiple users or processes from modifying the same resource simultaneously. This helps avoid conflicts and ensures data consistency.
3. **Plan Calculation:** Terraform uses the state file to calculate and display the difference between the desired configuration (defined in your Terraform code) and the current infrastructure state. This helps you understand what changes Terraform will make before applying them.
4. **Resource Metadata:** The state file stores metadata about each resource, such as unique identifiers, which is crucial for managing resources and understanding their relationships.

### Disadvantages of Storing Terraform State in Version Control Systems (VCS):

1. **Security Risks:** Sensitive information, such as API keys or passwords, may be stored in the state file if it's committed to a VCS. This poses a security risk because VCS repositories are often shared among team members.

2. **Versioning Complexity:** Managing state files in VCS can lead to complex versioning issues, especially when multiple team members are working on the same infrastructure.

## # Remote Backends

A remote backend in Terraform is used to store the state file outside your local machine, typically in a centralized and secure location.

### Why Use Remote Backends

- **Team Collaboration:**  
Multiple users can work on the same infrastructure using a shared state.
- **State Locking:**  
Prevents conflicts by locking the state during execution (e.g., using DynamoDB with S3).
- **Security:**  
Supports encryption and access control (IAM policies).
- **Reliability:**  
Protects against local file loss or corruption.

### Overcoming State File Issues using Remote Backend (S3)

To overcome issues like security risks, conflicts, and lack of locking in local state files, we use a remote backend like Amazon Web Services S3.

### Steps to Configure S3 Remote Backend

#### Step 1: Create an S3 Bucket

In **Amazon Web Services**, create an S3 bucket to store the Terraform state file.

Enable **versioning** and restrict access using IAM policies so the state is secure and recoverable.

#### Step 2: Create a DynamoDB Table (for Locking)

Create a DynamoDB table with a primary key (e.g., LockID).

This table will be used by Terraform to **lock the state file**, preventing multiple users from modifying it at the same time.

#### Step 3: Configure Backend in Terraform Code

Add the backend configuration in your `.tf` file:

```
terraform {
 backend "s3" {
 bucket = "my-terraform-state-bucket"
 key = "env/prod/terraform.tfstate"
 region = "ap-south-1"
 encrypt = true
 dynamodb_table = "terraform-lock-table"
 }
}
```

This tells Terraform:

- where to store the state (S3)

- how to secure it (encryption)
- how to handle locking (DynamoDB)

## Step 4: Initialize Terraform

Run:

```
terraform init
```

Terraform will:

- connect to S3
- create or use the existing state file
- enable locking using DynamoDB

## # State Locking

State locking is a mechanism in Terraform that prevents multiple users or processes from modifying the same state file at the same time.

### Why State Locking is Needed

Terraform state is a **single source of truth**.

If two people run `terraform apply` simultaneously:

- Both read the same state
- Both try to make changes
- This can lead to **conflicts, race conditions, or corrupted infrastructure**

State locking avoids this problem.

### How It Works

When you run commands like:

```
terraform apply
```

- Terraform **acquires a lock** on the state file
- No other user can modify the state during this time
- Once the operation completes, the **lock is released**

### For ex - State Locking with DynamoDB

## Step 1: Create an S3 Bucket (Store State File)

In **Amazon Web Services**, first create an S3 bucket.

**What you do:**

- Go to S3 → Create bucket
- Give a **unique name** (e.g., `my-terraform-state-bucket`)
- Enable **versioning**
- Block public access

### Why this step is important:

- Stores Terraform state centrally
- Versioning helps recover old state
- Prevents data loss and improves security

## Step 2: Create DynamoDB Table (Enable Locking)

### What you do:

Create a DynamoDB table using CLI or console:

```
aws dynamodb create-table \
 --table-name terraform-lock-table \
 --attribute-definitions AttributeName=LockID,AttributeType=S \
 --key-schema AttributeName=LockID,KeyType=HASH \
 --billing-mode PAY_PER_REQUEST
```

### Important Configuration:

- **Table Name:** terraform-lock-table
- **Primary Key:** LockID (String)

### Why this step is important:

- This table stores **lock information**
- Terraform writes a lock record when execution starts
- Prevents **parallel execution issues**

## Step 3: Configure Backend in Terraform

### What you do:

Add this in your Terraform file:

```
terraform {
 backend "s3" {
 bucket = "my-terraform-state-bucket"
 key = "env/prod/terraform.tfstate"
 region = "ap-south-1"
 encrypt = true
 dynamodb_table = "terraform-lock-table"
 }
}
```

### What each line means:

- **bucket:** Where state file is stored (S3)
- **key:** Path inside bucket (helps separate envs like dev/prod)
- **region:** AWS region
- **encrypt:** Secures state file
- **dynamodb\_table:** Enables locking using DynamoDB

## Step 4: Initialize Terraform

### Command:

```
terraform init
```

### What happens internally:

- Terraform connects to S3
- Creates/reads state file
- Links DynamoDB table for locking
- Backend is now active

## Step 5: How Locking Actually Works

When you run:

```
terraform apply
```

### Behind the scenes:

1. Terraform checks DynamoDB
2. If no lock exists → it creates a **lock entry**
3. Runs the operation
4. Deletes the lock after completion

## Step 6: What Happens in Conflict Scenario

### Example:

- User A runs `apply` → lock created
- User B runs `apply` → gets error

Error acquiring the state lock

### Why:

- DynamoDB already has a lock entry
- Prevents state corruption

## Step 7: Manual Unlock (If Needed)

If a lock is stuck (e.g., crash):

```
terraform force-unlock <LOCK_ID>
```

Use carefully — only when you're sure no one else is running Terraform.

### Final Flow

S3 → stores state

DynamoDB → locks state

Terraform → uses both for safe execution

## # Terraform File Provisioner

The file provisioner in Terraform is used to copy files or directories from your local machine to a remote resource (like an EC2 instance or VM) after it is created.

### Why Use File Provisioner?

- To **transfer configuration files**
- To **deploy scripts**
- To copy **application code** to a server

### Basic Syntax

```
resource "aws_instance" "example" {
 ami = "ami-123456"
 instance_type = "t2.micro"
 provisioner "file" {
 source = "local_file.txt"
 destination = "/home/ec2-user/remote_file.txt"
 }
 connection {
 type = "ssh"
 user = "ec2-user"
 private_key = file("key.pem")
 host = self.public_ip
 }
}
```

### Important Components

#### 1. source

- Path of file on **local machine**
- Example: "app.sh"

#### 2. destination

- Path on **remote server**
- Example: "/home/ec2-user/app.sh"

#### 3. connection block

Defines how Terraform connects to the remote resource:

- `type` → ssh / winrm
- `user` → OS username
- `private_key` → SSH key
- `host` → IP of instance

## **Goal: Copy file from your system → EC2**

### **Execution Flow:**

#### **Step 1: Terraform starts**

You run:

```
terraform apply
```

#### **Step 2: EC2 instance is created**

```
resource "aws_instance" "example" {
```

- Terraform calls AWS API
- EC2 instance gets launched
- Instance gets **public IP**

#### **Step 3: Terraform waits for SSH**

- It waits until EC2 is **reachable via SSH**
- Important: Security group must allow port 22

#### **Step 4: File provisioner starts**

```
provisioner "file" {
```

#### **Step 5: Read local file**

```
source = "local/path/to/localfile.txt"
```

- Terraform reads file from your **local machine**

#### **Step 6: Establish SSH connection**

```
connection {
 type = "ssh"
```

- Uses:
  - username (ec2-user)
  - private key
  - EC2 public IP

#### **Step 7: Copy file to EC2**

```
destination = "/path/on/remote/instance/file.txt"
```

- File is transferred via SSH (SCP internally)

### **Final Result:**

File is now inside EC2

## # Terraform Remote-Exec Provisioner

The **remote-exec provisioner** in Terraform is used to **execute commands on a remote machine** (like EC2, VM) **after it is created**.

### Why Use Remote-Exec?

- Install packages (e.g., nginx, docker)
- Configure server
- Run scripts remotely
- Perform post-deployment setup

### Goal: Run commands on EC2

#### Execution Flow:

#### Step 1: Terraform apply

`terraform apply`

#### Step 2: EC2 instance created

Same as before

#### Step 3: Wait for SSH access

- Instance must be fully ready

#### Step 4: Remote-exec starts

```
provisioner "remote-exec" {
```

#### Step 5: Establish SSH connection

```
host = aws_instance.example.public_ip
```

- Terraform logs into EC2

#### Step 6: Execute commands one by one

```
inline = [
 "sudo yum update -y",
 "sudo yum install -y httpd",
 "sudo systemctl start httpd",
]
```

Execution order:

1. Update packages
2. Install Apache
3. Start Apache



## Step 7: Commands run inside EC2

- Same as if you manually SSH and run them

### Final Result:

Apache installed

Web server running

## # Terraform Local-Exec Provisioner

The local-exec provisioner in Terraform is used to run commands on your local machine (where Terraform is executed), not on the remote resource.

### Why Use Local-Exec?

- Call scripts after resource creation
- Trigger external tools (CLI, APIs)
- Send notifications (Slack, email)
- Update local files/logs

### Goal: Run command on your local machine

## Step 1: Terraform apply

```
terraform apply
```

## Step 2: Null resource created

```
resource "null_resource" "example" {
```

- No real infrastructure
- Just a placeholder

## Step 3: Trigger evaluated

```
triggers = {
 always_run = "${timestamp()}"
}
```

- `timestamp()` changes every time
- Forces Terraform to re-run provisioner

## Step 4: Local-exec starts

```
provisioner "local-exec" {
```

## Step 5: Command runs locally

```
command = "echo 'This is a local command'"
```

- Runs on:

- Your laptop
- Or CI/CD server

## Step 6: Output shown in terminal

This is a local command

### Final Result:

Command executed locally

## # Terraform Workspaces

Terraform Workspaces allow you to manage multiple environments (like dev, staging, prod) using the same Terraform configuration, without duplicating code.

### What is a Workspace?

A **workspace** in Terraform is a **separate state file** within the same configuration.

Each workspace maintains its **own state**, so resources don't conflict

### Why Use Workspaces?

- Avoid duplicating code for different environments
- Isolate environments (dev, test, prod)
- Manage infrastructure efficiently
- Reduce errors and conflicts

### Default Workspace

Terraform starts with a default workspace called: **default**

### Common Commands

#### 1. List Workspaces

```
terraform workspace list
```

#### 2. Create New Workspace

```
terraform workspace new dev
```

#### 3. Switch Workspace

```
terraform workspace select dev
```

#### 4. Show Current Workspace

```
terraform workspace show
```

#### 5. Delete Workspace

```
terraform workspace delete dev
```

**DEMO: We are using Terraform Workspaces to deploy different EC2 configurations for different environments**

Environment	Instance Type
dev	small (cheap)
stage	medium
prod	large (powerful)

#### Step 1: Write Common Terraform Code

You write **one reusable code**

- Provider (AWS)
- Variables
- Module
- Resource

No duplication for dev/stage/prod

#### Step 2: Define AMI

```
terraform.tfvars
```

```
ami = "ami-053b0d53c279acc90"
```

Same AMI used everywhere

#### Step 3: Define Environment-Based Config

```
variable "instance_type" {
 type = map(string)
 default = {
 dev = "t2.micro"
 stage = "t2.medium"
 prod = "t2.xlarge"
 }
}
```

You are saying:

- If dev → small instance
- If stage → medium
- If prod → large

### Step 4: Use Workspace Dynamically

```
instance_type = lookup(var.instance_type, terraform.workspace, "t2.micro")
```

This line connects:

- Workspace → Instance Type

### Step 5: Create Module

```
module "ec2_instance" {
 source = "../modules/ec2_instance"
 ami = var.ami
 instance_type = lookup(...)
}
```

You are passing values to module

### Step 6: Inside Module

```
resource "aws_instance" "example" {
 ami = var.ami
 instance_type = var.instance_type
}
```

Actual EC2 gets created here

## Now Execution Steps

### Step 7: Initialize Terraform

```
terraform init
```

Downloads AWS provider

Prepares project

### Step 8: Create Workspaces

```
terraform workspace new dev
terraform workspace new stage
terraform workspace new prod
```

Now you have 3 environments

### Step 9: Select Workspace

```
terraform workspace select dev
```

Now Terraform knows:

```
terraform.workspace = "dev"
```

## Step 10: Apply Terraform

terraform apply

What happens internally:

1. Reads workspace → dev
2. Looks into map:

dev → t2.micro

3. Passes value to module
4. Creates EC2:

instance\_type = t2.micro

## Repeat for Other Environments

### For stage:

terraform workspace select stage

terraform apply

EC2 → t2.medium

### For prod:

terraform workspace select prod

terraform apply

EC2 → t2.xlarge

## Step 11: State Separation

Each workspace has its **own state file**

Workspace	State
dev	separate
stage	separate
prod	separate
No conflict	
Safe deployments	

## Final Result

Using ONE codebase you created:

- Dev EC2 (small)
- Stage EC2 (medium)
- Prod EC2 (large)

## # Vault Integration

Vault Integration means connecting HashiCorp Vault with applications, tools, or infrastructure to securely store, manage, and access secrets like:

- API keys
- Database credentials
- Tokens
- Certificates

Instead of hardcoding secrets, Vault provides them **dynamically and securely**.

## Why Do We Need Vault Integration?

### Problems without Vault:

- Secrets stored in code (security risk )
- Hard to rotate credentials
- No audit tracking

### With Vault:

- Centralized secret management
- Dynamic secrets (auto-expire)
- Encryption + access control
- Full audit logs

### How Vault Integration Works (Flow)

### Step-by-Step Flow:

1. Application/User requests secret
2. Authenticates to Vault
  - AWS IAM / Kubernetes / AppRole / Token
3. Vault verifies identity
4. Vault checks policies (RBAC)
5. Vault generates or fetches secret
6. Secret is returned (with TTL)

Secrets are **temporary**, not permanent.

### Common Authentication Methods

- **AppRole** → Used in CI/CD pipelines
- **Kubernetes Auth** → Pods authenticate using service accounts
- **AWS IAM Auth** → EC2/Lambda-based authentication
- **Token Auth** → Simple but less secure

## ◆ VAULT INTEGRATION (EC2 + Vault + Terraform)

### STEP 1: Create EC2 (Ubuntu)

Using **Amazon Web Services**

## Steps:

1. Go to EC2 Dashboard
2. Click **Launch Instance**
3. Choose **Ubuntu LTS AMI**
4. Select instance type → **t2.micro** (free tier)
5. Create/Select Key Pair
6. Network:
  - Allow **SSH (22)**
  - Allow **Custom TCP (8200)** ← for Vault
7. Launch instance

## Connect:

```
ssh -i key.pem ubuntu@<public-ip>
```

## STEP 2: Install Vault on EC2

Installing HashiCorp Vault

### 2.1 Update & Install GPG

```
sudo apt update && sudo apt install -y gpg
```

### 2.2 Add HashiCorp GPG Key

```
wget -O- https://apt.releases.hashicorp.com/gpg | \
gpg --dearmor | \
sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg
```

### 2.3 Add Repository

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update
```

### 2.4 Install Vault

```
sudo apt install -y vault
```

## STEP 3: Start Vault

```
vault server -dev -dev-listen-address="0.0.0.0:8200"
```

### Important Output:

- Root Token
- Vault Address

### 3.1 Set Environment Variables (NEW TERMINAL)

```
export VAULT_ADDR='http://<EC2-IP>:8200'
export VAULT_TOKEN='root-token'
```

### 3.2 Test Vault

```
vault status
```

## STEP 4: Store Secrets in Vault

### 4.1 Enable KV Secrets Engine

```
vault secrets enable -path=secret kv-v2
```

### 4.2 Add Secret

```
vault kv put secret/myapp \
username="admin" \
password="mypassword"
```

### 4.3 Read Secret

```
vault kv get secret/myapp
```

## STEP 5: Enable AppRole Authentication

### 5.1 Enable AppRole

```
vault auth enable approle
```

### 5.2 Create Policy

```
vault policy write terraform - <<EOF
path "secret/data/myapp" {
 capabilities = ["read"]
}
EOF
```

### 5.3 Create Role

```
vault write auth/approle/role/terraform-role \
token_policies="terraform" \
token_ttl=1h \
token_max_ttl=4h
```

### 5.4 Get Role ID

```
vault read auth/approle/role/terraform-role/role-id
```

### 5.5 Get Secret ID

```
vault write -f auth/approle/role/terraform-role/secret-id
Save:
```

- role\_id
- secret\_id



## STEP 6: Integrate with Terraform

### 6.1 Install Terraform

```
sudo apt install -y terraform
```

### 6.2 Terraform Code

```
provider "vault" {
 address = "http://<EC2-IP>:8200"
}
data "vault_kv_secret_v2" "example" {
 mount = "secret"
 name = "myapp"
}
output "username" {
 value = data.vault_kv_secret_v2.example.data["username"]
}
```

### 6.3 Authenticate via AppRole

```
export VAULT_ADDR=http://<EC2-IP>:8200
vault write auth/approle/login \
role_id=<role_id> \
secret_id=<secret_id>
Copy client token:
```

```
export VAULT_TOKEN=<client-token>
```

### 6.4 Run Terraform

```
terraform init
terraform apply
```

## COMPLETE FLOW

Flow:

1. Terraform → authenticates using AppRole
2. Vault → validates role\_id & secret\_id
3. Vault → issues temporary token
4. Terraform → reads secret
5. Secret used → NOT stored in code

## # Ways to Secure Terraform -

Terraform allows you to mark variables/outputs as sensitive so they are hidden in logs/output.

### Example:

```
variable "aws_access_key_id" {
 sensitive = true
}
```

### What happens:

- Value will NOT be shown in:

- terraform plan
- terraform apply
- But still stored in state file

#### Why use it:

- Prevent accidental exposure in logs

#### Limitation:

- Does NOT encrypt data
- Only hides from UI

Interview Line:

“Sensitive = hides output, but not a complete security solution.”

## 2 Secret Management System

Using tools like:

- HashiCorp Vault
- AWS Secrets Manager

#### What it means:

Instead of hardcoding secrets in Terraform, store them in Vault and fetch dynamically.

#### Example:

```
data "vault_generic_secret" "aws_access_key" {
 path = "secret/aws/access_key_id"
}
variable "aws_access_key_id" {
 value = data.vault_generic_secret.aws_access_key.data["value"]
}
```

#### What happens:

1. Terraform calls Vault
2. Vault authenticates
3. Returns secret securely

#### Why best practice:

- No secrets in code
- Dynamic secrets
- Centralized security

Interview Line:

“We integrate Terraform with Vault to avoid hardcoding credentials.”

### 3 Remote Backend

#### What it means:

Terraform state file (`terraform.tfstate`) contains:

- passwords
- API keys
- infrastructure details

So we store it securely in remote backend.

#### Example (S3 Backend):

```
terraform {
 backend "s3" {
 bucket = "my-tf-state"
 key = "state.tfstate"
 region = "ap-south-1"
 encrypt = true
 }
}
```

#### What happens:

- State stored in S3
- Encryption enabled
- Can add DynamoDB for locking

#### Why:

- Secure storage
- Team collaboration
- Prevent state corruption

Interview Line:

“State file is sensitive, so we store it in encrypted remote backend like S3.”

### 4 Environment Variables

#### What it means:

Store secrets in OS instead of code.

#### Example:

```
export AWS_ACCESS_KEY_ID=your_key
export AWS_SECRET_ACCESS_KEY=your_secret
```

Terraform automatically reads:

```
provider "aws" {}
```

**What happens:**

- Terraform picks values from environment
- No need to write in `.tf` file

**Why:**

- Avoid hardcoding
- Easy for CI/CD

**Limitation:**

- Not centralized
- Can leak if system compromised

Snehal B | Terraform Notes